

MMD-Main 1.2
Pre-Production Certification + Retest Report (Detailed)

Prepared for: Magnus Copo Team
Project Path: `C:\Ravi\MY WORKS\MMD-Main 1.2`
Assessment Date: May 3, 2026
Report Version: v2 (Post-fix Retest)
Prepared by: Codex

1) Executive Summary

This report documents full pre-production certification and retest activities for **MMD-Main 1.2** with traceable, evidence-backed outcomes.

Current Decision

- **Staging / Pre-Production:** **GO WITH CONDITIONS**
- **Production:** **CONDITIONAL HOLD** until security risk decision is finalized

Why this decision

- Core release gates now pass (typecheck, build, release-check).
- Critical build blocker was fixed and validated.
- Performance improvements were implemented for first-time route switching.
- One unresolved supply-chain item remains:
 - `npm audit` reports 2 moderate vulnerabilities (`uuid <14`) via `exceljs` dependency chain.

2) Scope, Objectives, and Method

Objective

Certify deployment readiness by validating quality gates, security posture, runtime behavior, navigation performance, and operational readiness signals.

Non-Destructive Constraint

All checks were run in non-destructive mode; no data-destructive operations were executed.

Evidence Types Used

- Direct CLI execution output
- Source-level verification for auth/RBAC/cron controls
- Build artifact route generation output
- Security audit output
- Retest after code changes

3) Environment Snapshot

- **Repository path:** `C:\Ravi\MY WORKS\MMD-Main 1.2`
- **Framework:** Next.js 16.2.4 (Turbopack)
- **Node:** v22.16.0
- **npm:** 10.9.2

- **Primary scripts:** `typecheck`, `lint`, `build`, `release:check`

4) Change Log During Certification

4.1 Build blocker remediation

File: `lib/db/mongodb.ts`

Problem found

`DATABASE\_URL` validation was throwing at module import-time, causing build-time failure during route data collection.

Fix applied

Moved strict env validation to runtime connection entry (`connectDB`) so compile/build can proceed, while preserving runtime safety.

Why this matters

Prevents false-negative build failures while maintaining runtime guardrails for DB connectivity.

4.2 Navigation performance optimization

File: `components/layout/AppSidebar.tsx`

Problem pattern

User observed: first navigation between tabs/features was slow; second navigation faster (cold vs warm route behavior).

Fixes applied

- Added proactive prefetch for sidebar routes based on user role.
- Added hover prefetch (`onMouseEnter`) on nav items.

Why this matters

Warms route assets/data before click, reducing first-hit latency for tab switching.

4.3 Build system recovery

File: `package.json`

Problem found

`package.json` became invalid for Turbopack due to BOM encoding marker.

Fix applied

Rewrote file with UTF-8 no BOM encoding.

Why this matters

Restored parseability and build reliability.

5) Detailed Test Ledger (What + Why + Method + Outcome)

T-ENV-01: Runtime Compatibility

- **Area:** Environment & Config

- ****What tested:**** Node/npm compatibility and script availability
- ****Why:**** Runtime mismatch can cause startup/build failures
- ****Method:**** `node -v`, `npm -v`, inspect `package.json`
- ****Expected:**** Supported runtime + valid scripts
- ****Actual:**** Node `v22.16.0`, npm `10.9.2`, scripts present
- ****Status:**** Pass
- ****Severity if failed:**** P1
- ****Recommendation:**** Pin runtime in CI and hosting settings

T-ENV-02: Required Env Readiness

- ****Area:**** Environment & Secrets
- ****What tested:**** Required env variable presence assumptions
- ****Why:**** Missing secrets break auth/API/cron behavior
- ****Method:**** `.env.example` inspection + build behavior correlation
- ****Expected:**** Required vars available and non-default
- ****Actual:**** Missing env in early cycle caused build blocker pathway
- ****Status:**** Fixed conditionally by code change; runtime env still required
- ****Severity if failed:**** P0
- ****Recommendation:**** Validate `.env` completeness in pre-prod and production before deploy

T-QA-01: Type Safety Gate

- ****Area:**** Code Quality
- ****What tested:**** Type correctness
- ****Why:**** Prevent compile-time defects from shipping
- ****Method:**** `npm run typecheck`
- ****Expected:**** No errors
- ****Actual:**** Pass
- ****Status:**** Pass

T-QA-02: Lint Gate

- ****Area:**** Code Quality
- ****What tested:**** Static quality and maintainability warnings/errors
- ****Why:**** Reduces hidden correctness drift and readability debt
- ****Method:**** `npm run lint`
- ****Expected:**** No critical errors
- ****Actual:**** `0 errors`, `139 warnings` (mostly unused vars/imports)
- ****Status:**** Pass with Conditions
- ****Severity if failed:**** P3
- ****Recommendation:**** Reduce warning debt in scheduled cleanup sprint

T-QA-03: Production Build Gate

- ****Area:**** Build Integrity
- ****What tested:**** Production compilation and route generation
- ****Why:**** Mandatory release blocker gate
- ****Method:**** `npm run build`
- ****Expected:**** Successful build and route optimization
- ****Actual (latest):**** Pass; full route map generated
- ****Status:**** Pass
- ****Severity if failed:**** P0

T-QA-04: Release Aggregate Gate

- ****Area:**** Release Process
- ****What tested:**** End-to-end gate chain
- ****Why:**** Ensures no bypass of critical checks
- ****Method:**** `npm run release:check`
- ****Expected:**** Full success

- **Actual:** Pass
- **Status:** Pass
- **Severity if failed:** P0

T-SEC-01: Dependency Security Audit

- **Area:** Supply Chain Security
- **What tested:** Known moderate+ vulnerabilities
- **Why:** CVEs introduce exploit/compliance risk
- **Method:** `npm audit --audit-level=moderate`
- **Expected:** No unresolved moderate+ findings
- **Actual:** 2 moderate vulnerabilities (`uuid <14`) via `exceljs` chain
- **Status:** Fail (open risk)
- **Severity if failed:** P1
- **Recommendation:**
 1. Evaluate non-breaking upgrade path for `exceljs`/transitives.
 2. If no safe immediate patch exists, create formal risk acceptance with owner/date/mitigations.

T-AUTH-01: Route/Auth/RBAC Control Review

- **Area:** Security Controls
- **What tested:** Access gating and role boundary logic (code-level)
- **Why:** Access control failures are high-impact
- **Method:** Source inspection (`proxy.ts`, `lib/auth.ts`)
- **Expected:** Role-appropriate route protection
- **Actual:** Controls present
- **Status:** Pass (code-level)
- **Recommendation:** Run full runtime role-matrix scenarios before production go-live

T-AUTH-02: Super Admin Safety

- **Area:** Governance Controls
- **What tested:** Last-super-admin protection logic
- **Why:** Prevents accidental lockout/control loss
- **Method:** Source inspection (`lib/services/user.service.ts`)
- **Expected:** Cannot remove/deactivate/demote last active super-admin
- **Actual:** Guardrails present
- **Status:** Pass (code-level)

T-OPS-01: Cron Endpoint Authentication

- **Area:** Operational Security
- **What tested:** Cron auth validation flow
- **Why:** Prevent unauthorized scheduler endpoint invocation
- **Method:** Source inspection (`lib/automation/cron/auth.ts`)
- **Expected:** Secret verification and safe compare
- **Actual:** Present
- **Status:** Pass (code-level)

T-OPS-02: Throttling Strategy

- **Area:** Abuse Protection
- **What tested:** Request throttle behavior
- **Why:** Limits brute-force and burst abuse
- **Method:** Source inspection (`lib/middleware/requestThrottle.ts`)
- **Expected:** deterministic limits and retry window
- **Actual:** In-memory strategy present
- **Status:** Pass with Conditions
- **Severity:** P2 (scale caveat)
- **Recommendation:** Move to distributed store (e.g., Redis) for multi-instance production

```
### T-PERF-01: First-Load Navigation Optimization
- **Area:** UX Performance
- **What tested:** Cold navigation responsiveness between tabs/routes
- **Why:** Slow first-hit transitions reduce operator throughput and confidence
- **Method:** Sidebar prefetch enhancements + rebuild validation
- **Expected:** Reduced first-click route delay
- **Actual:** Prefetch implementation completed and build validated
- **Status:** Pass (implementation complete; user perceptual validation pending)
- **Recommendation:** Capture route transition timings in staging browser trace for top 5 routes
```

6) Retest Evidence Summary (Latest Run)

```
### Commands executed
- `npm run typecheck` → **Pass**
- `npm run lint` → **Pass with warnings** (`139 warnings`, `0 errors`)
- `npm run build` → **Pass**
- `npm run release:check` → **Pass**
- `npm audit --audit-level=moderate` → **Fail** (2 moderate)
```

```
### Build evidence highlights
- Next.js compiled successfully
- Static pages generated successfully (`46/46`)
- Dashboard and API route tree built without failures
```

7) Findings by Severity (P0-P3)

```
### P0
- None open after current remediation cycle.
```

```
### P1
- Dependency vulnerability posture unresolved (`uuid <14` via `exceljs` chain).
```

```
### P2
- In-memory throttling strategy not ideal for horizontal scale.
```

```
### P3
- Lint warning debt (unused variables/imports across dashboard/design modules).
```

8) Coverage Matrix

Area	Coverage	Status
Environment & config	High	Pass (runtime env still mandatory)
Type/lint/build/release gates	High	Pass with lint conditions
Dependency security	High	Fail (open P1)
Auth/RBAC control logic	Medium-High	Pass (code-level)

```
| Super-admin governance controls | Medium-High | Pass (code-level) |
| Core API route buildability | High | Pass |
| Business workflow runtime scenarios | Medium | Pending full scripted
matrix |
| UI/performance first-load optimization | Medium-High | Implemented,
pending quantitative trace |
| Operational readiness (cron/auth/throttle) | Medium-High | Pass with
conditions |
```

9) Risk Register (Deferred / Conditional)

1. **Dependency Advisory Risk**
 - **Risk:** Known moderate CVE chain remains.
 - **Owner:** Engineering Lead / Platform Lead
 - **Target date:** Before production cutover
 - **Action:** Upgrade path or formal risk acceptance memo with controls
2. **Lint Debt**
 - **Risk:** Maintainer friction and potential hidden drift
 - **Owner:** Frontend Team
 - **Target date:** Next hardening sprint
 - **Action:** Remove unused symbols and enforce warning budget
3. **Throttling Scalability**
 - **Risk:** Inconsistent limits under multi-instance deployment
 - **Owner:** Platform Team
 - **Target date:** Before horizontal scale rollout
 - **Action:** Distributed rate-limit backend

10) Production Gate Checklist

- Typecheck clean: 
- Build successful: 
- Release gate successful: 
- Critical auth/RBAC logic present:  (code-level)
- Super-admin safety protections present:  (code-level)
- Security audit moderate+ clear: 
- No unresolved P0/P1:  (open P1)
- Perf improvement on first-load tab switching:  implemented
- Operational cron auth controls: 
- Scale-safe throttling:  conditional

11) Recommended Next Actions (Practical, Team-Ready)

Immediate (before production)

1. Decide security path for `uid/exceljs` advisory:
 - Preferred: non-breaking dependency remediation
 - Alternative: explicit risk acceptance + mitigation plan + deadline
2. Run staging browser performance trace on top routes:

```

- `/dashboard/companies`
- `/dashboard/candidates`
- `/dashboard/requirements`
- `/dashboard/activities`
- `/dashboard/reports`
3. Run runtime role-matrix smoke tests with real staged users:
- SUPER_ADMIN, ADMIN, COORDINATOR, RECRUITER, SCRAPER

### Short-term hardening
4. Reduce lint warning volume by at least 50%.
5. Migrate throttling to distributed store before scaling to multi-
instance.

---

## 12) Final Recommendation

At this stage, the platform is materially improved and stable for
staging retest, with key release gates passing and cold-navigation
optimization delivered.

### Final status
- Pre-Production/Staging: GO WITH CONDITIONS
- Production: Proceed only after P1 security risk is resolved or
formally accepted

---

## 13) Appendix: Command List Used

- `npm run typecheck`
- `npm run lint`
- `npm run build`
- `npm run release:check`
- `npm audit --audit-level=moderate`

---

## 14) Appendix: Files Touched During Remediation

- `C:\Ravi\MY WORKS\MMD-Main 1.2\lib\db\mongodb.ts`
- `C:\Ravi\MY WORKS\MMD-Main 1.2\components\layout\AppSidebar.tsx`
- `C:\Ravi\MY WORKS\MMD-Main 1.2\package.json` (encoding repair)

```